Acknowledgement

I would like to thank **Unified Mentor** for giving me the opportunity to work on the *Operation Scheduler for Hospital Management* project as part of my internship. This internship helped me gain practical experience in web development and gave me a clear understanding of how real-world healthcare systems are designed and implemented using technologies like HTML, CSS, JavaScript, and Firebase.

I am sincerely grateful to the mentors and support team at Unified Mentor for their continuous guidance and support throughout the internship. Their suggestions, feedback, and structured project guidelines helped me understand the practical challenges involved in building a hospital operation scheduling system and improved my overall technical and problem-solving skills.

I would also like to thank my faculty members and peers for their encouragement and support during the completion of this project. Their motivation helped me stay focused and complete the internship work successfully within the given timeline.

Finally, I express my sincere gratitude to everyone who directly or indirectly supported me, **Upasana Nayak**, in completing this internship project successfully.

Abstract

The *Operation Scheduler for Hospital Management* is a healthcare web application developed to make the process of scheduling operation theaters (OT) and managing surgical activities easier and more organized. Hospitals often face difficulties in managing surgery schedules, doctor availability, and operation theater usage manually, which can lead to confusion and scheduling conflicts. This system is designed to provide a simple and efficient solution to manage these activities in a structured way.

The application allows administrators to schedule surgeries, manage doctor and patient details, assign operation theaters, and monitor surgical activities in real time. Users can view doctor information and surgical schedules easily through a user-friendly interface. The system is developed using HTML, CSS, and JavaScript for the front-end and Firebase for database and backend support, ensuring smooth data handling and easy system management.

The main goal of this system is to improve hospital workflow, reduce manual work, and create a reliable and easy-to-use operation scheduling platform. The modular design of the application makes it easy to maintain, scalable for future improvements, and suitable for real-world hospital environments.

Content

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives of the Project
5. Scope of the Project
6. Technologies and Tools Used
7. System Requirements
8. System Architecture
9. System Modules (Admin & User)
10. System Workflow
11. Database Design (mongodb)
12. ER Diagram
13. Interface Design / Screenshots
14. Implementation Details
15. GitHub Repository and Code Structure
16. Testing and Test Cases
17. Challenges Faced
18. Future Enhancements
19. Conclusion
20. References
21. GitHub Repository Link

Introduction

The healthcare sector is one of the most important fields in modern society because it directly affects patient safety, treatment quality, and overall well-being. Hospitals manage a large number of activities every day, including patient care, doctor coordination, surgical procedures, and resource management. Among these, operation theater (OT) scheduling is one of the most critical and complex tasks, as it requires proper coordination between doctors, patients, nurses, anesthesiologists, operation rooms, and medical equipment within a limited time. If the scheduling process is not handled properly, it can lead to delayed surgeries, poor resource utilization, and inconvenience for both patients and hospital staff.

In many hospitals, operation theater scheduling is still handled manually or through basic systems such as spreadsheets or static records. This makes the process slow and increases the chances of human error. Hospital administrators often face challenges in managing doctor availability, assigning operation theaters, handling emergency cases, and maintaining proper surgical records. Manual scheduling may result in overlapping surgery timings, underutilized operation theaters, and difficulty in tracking patient and surgical information. As hospitals continue to grow and handle more patients, the need for an automated and organized scheduling system becomes essential.

With the advancement of web technologies and modern database systems, hospitals are gradually shifting towards digital solutions to improve their management processes. A web-based operation scheduling system can help administrators manage surgical activities more efficiently by providing real-time access to information and reducing manual workload. Such systems improve coordination among hospital staff, ensure proper resource allocation, and maintain accurate surgical records in a secure and structured manner.

As part of the internship at Unified Mentor, the *Operation Scheduler for Hospital Management* system was developed to address these challenges. The main aim of this system is to simplify and automate the process of operation theater scheduling and surgical management in hospitals. The application provides a structured platform where administrators can manage doctor details, patient information, surgery schedules, and operation theater assignments easily. At the same time, users can view doctor and surgical information through a simple and user-friendly interface.

The system is built using HTML, CSS, and JavaScript for the front-end, while MongoDB is used as the database to store and manage hospital data. MongoDB provides a flexible and scalable way to handle large amounts of data, including doctor records, patient details, and operation schedules. This ensures that the system can store data securely, retrieve information quickly, and maintain proper organization of hospital records.

The introduction of this system helps improve hospital workflow by reducing scheduling conflicts and making the entire operation scheduling process more organized and efficient. It also ensures better coordination among hospital staff and provides real-time access to surgical data, which helps in making quick and accurate decisions.

The main goals of this project are:

- To automate operation theater scheduling
- To manage doctor and patient information efficiently
- To reduce manual errors and scheduling conflicts
- To improve hospital workflow and coordination
- To provide a simple and user-friendly web interface
- To ensure secure and structured data storage using MongoDB
- To develop a scalable and maintainable healthcare management system

Project Overview

The *Operation Scheduler for Hospital Management* is a web-based healthcare application designed to make the process of scheduling surgeries and managing operation theater activities easier and more organized. The system provides a centralized platform where hospital administrators can manage doctor availability, patient records, operation theater assignments, and surgical schedules in a structured and efficient manner.

Hospitals perform multiple surgeries every day, and managing these operations manually can create several challenges. Administrators need to coordinate between different departments, manage emergency cases, assign operation theaters, and maintain proper records of surgical activities. Without an automated system, this process becomes time-consuming and increases the chances of errors. The proposed system solves this problem by introducing a digital platform that helps manage all operation-related activities through a single interface.

The application allows administrators to add doctor details, manage patient information, schedule surgeries, assign operation theaters, and monitor surgical activities in real time. Users can log in to the system and view doctor information and surgical schedules, which improves transparency and accessibility. The use of MongoDB as the database ensures that all hospital data is stored securely and can be accessed quickly whenever needed.

The system follows a modular design approach, which means that different parts of the system are developed separately and then integrated together. This makes the system easy to maintain, test, and upgrade in the future. Each module performs a specific function and contributes to the overall working of the operation scheduling system.

The major components of the system include:

1. Admin Module
This module allows administrators to control and manage all hospital operation activities.
Main functions:

- Add and manage doctor details
- Add and manage patient information
- Schedule surgeries
- Assign operation theater IDs
- Update surgical records
- Monitor OT activities
- Add operation reports and remarks

2. User Module
This module allows users to access operation-related information in a secure manner.
Main functions:

- User registration and login
- View doctor details
- View surgical schedules
- Access operation information

3. Operation Scheduling Module
This is the core part of the system where surgical operations are planned and managed.
Key features:

- Assign operation theater number
- Schedule surgery date and time
- Add anesthesia and anesthesiologist details
- Track pre-operative and post-operative events
- Add surgical reports and remarks
- Manage required drugs and instruments

4. Database System (MongoDB)
MongoDB is used to store and manage all hospital data efficiently.
Main functions:

- Store doctor and patient records
- Manage operation schedules

- Maintain surgical and user data
- Provide fast data retrieval and updates
- Ensure secure and structured data storage

5. System Implementation
The system is developed using modern web technologies and follows standard software development practices.
Key aspects:

- Modular coding structure
- Secure data handling
- GitHub repository management
- Testing and validation
- Deployment planning

Problem Statement

Hospitals handle a large number of surgical procedures every day, and managing operation theaters (OT) efficiently is one of the most important responsibilities of hospital administration. Operation scheduling involves coordinating doctors, patients, anesthesiologists, nurses, operation rooms, and medical equipment within a limited time. If this process is not properly managed, it can lead to delays in surgeries, poor coordination among staff, and inefficient use of hospital resources. In many hospitals, operation scheduling is still handled manually or with basic systems, which makes the entire process slow and prone to errors.

Manual scheduling creates several practical challenges for hospital administrators. They need to track doctor availability, assign operation theaters, manage patient records, and handle emergency surgeries at the same time. Without a proper digital system, it becomes difficult to maintain accurate records and update schedules quickly. This often results in overlapping surgery timings, delays in operations, and confusion among hospital staff. In emergency situations, the lack of real-time data can make it harder to find available operation theaters or doctors, which may affect patient care and hospital efficiency.

Another major problem is the lack of a centralized system for storing and managing surgical information. Doctor details, patient records, operation schedules, anesthesia information, and surgical reports are often stored in separate files or documents. This makes it difficult to access information quickly and increases the risk of data loss or mismanagement. Administrators may spend extra time searching for records, updating schedules, or

coordinating with different departments, which reduces overall productivity and slows down hospital operations.

Some of the common problems faced in traditional operation scheduling systems include:

- Manual and time-consuming scheduling process
- Difficulty in managing doctor and patient records
- Overlapping surgery schedules and operation theater conflicts
- Lack of real-time monitoring of OT activities
- Poor coordination between hospital departments
- Delay in updating surgical and operation records
- Difficulty in handling emergency surgeries and last-minute changes
- Risk of data mismanagement or loss due to manual record-keeping

Because of these issues, hospitals need a reliable and automated system that can manage operation scheduling in a structured and efficient way. A web-based operation scheduling system can help administrators manage surgical activities easily by providing a centralized platform for storing and accessing hospital data. It can also reduce manual work, improve coordination among staff, and ensure proper utilization of operation theaters and medical resources.

The *Operation Scheduler for Hospital Management* system is developed to solve these problems by providing a simple and organized web-based solution. The system allows administrators to manage doctor details, patient information, and operation schedules in one place while ensuring secure and structured data storage using MongoDB. By automating the scheduling process, the system helps hospitals reduce errors, improve efficiency, and maintain better control over surgical operations.

Objectives of the Project

The main objective of the *Operation Scheduler for Hospital Management* project is to develop a web-based system that makes operation theater scheduling easier, faster, and more organized for hospitals. The system is designed to reduce manual work and provide a structured platform where administrators can manage surgical activities efficiently. By using modern web technologies and MongoDB as the database, the project aims to create a reliable and user-friendly solution for handling hospital operation scheduling.

The system focuses on improving the overall workflow of hospitals by providing a centralized platform to manage doctor details, patient records, and operation schedules. It ensures that all surgical information is stored securely and can be accessed easily whenever needed. The project also aims to reduce scheduling conflicts, improve coordination between hospital staff, and provide a simple interface that can be used without technical difficulty.

The key objectives of this project are listed below:

1. To automate operation theater scheduling

- Reduce manual scheduling work
- Assign operation theaters and surgery timings efficiently
- Avoid scheduling conflicts and delays

2. To manage doctor and patient information in an organized way

- Store doctor and patient records in a structured database
- Provide easy access to surgical and medical information
- Maintain accurate and updated data using MongoDB

3. To improve hospital workflow and coordination

- Provide a centralized platform for managing surgical activities
- Improve coordination between administrators and hospital staff
- Allow real-time monitoring of operation theater usage

4. To reduce human errors and increase efficiency

- Minimize errors caused by manual record-keeping
- Ensure accurate scheduling and data management
- Improve overall operational efficiency

5. To develop a user-friendly and secure system

- Provide a simple and easy-to-use interface
- Ensure secure login and data handling

- Protect hospital information through structured database storage

6. To create a scalable and maintainable system

- Use modular design for easy maintenance and future upgrades
- Ensure the system can handle increasing hospital data
- Make the system suitable for real-world healthcare environments

7. To implement modern technologies in healthcare management

- Use HTML, CSS, and JavaScript for front-end development
- Use MongoDB for database management
- Follow proper coding and deployment practices

Scope of the Project

The *Operation Scheduler for Hospital Management* project focuses on developing a web-based system that helps hospitals manage operation theater scheduling in a more organized and efficient way. The main scope of this project is to create a platform where administrators can easily manage doctors, patients, and surgical schedules without relying on manual records or traditional scheduling methods. The system is designed to simplify the overall operation scheduling process and improve coordination between different hospital departments involved in surgical activities.

This project mainly covers the management of operation theater scheduling and related hospital activities. It allows administrators to add and manage doctor details, store patient information, schedule surgeries, assign operation theaters, and maintain surgical records in a structured database using MongoDB. By using a web-based system, hospital staff can access and update information easily, which helps reduce manual work and improves the overall workflow of the hospital.

The scope of the project is limited to operation scheduling and basic hospital management related to surgical activities. It does not cover all hospital operations such as billing, pharmacy management, laboratory systems, or appointment booking. The main aim is to build a reliable operation scheduling system that can serve as a foundation for a complete hospital management system in the future. This focused approach ensures that the system remains simple, efficient, and easy to implement.

The system is designed to handle the essential functions required for operation theater management. These include managing doctor availability, storing patient records, scheduling surgeries, assigning operation theaters, and maintaining surgical reports. The use of MongoDB ensures that all hospital data is stored in a structured and secure manner, making it easier to retrieve and update information whenever needed.

The key scope of the project includes:

- Managing doctor details and availability
- Managing patient information and surgical records
- Scheduling surgeries and assigning operation theaters
- Maintaining anesthesia and surgical reports
- Providing secure login and access for users and administrators
- Storing hospital data using MongoDB database
- Providing a web-based interface for easy access and management
- Supporting future expansion and system upgrades

At the same time, the project has certain limitations to keep the system practical and manageable during the internship period. The system does not include advanced features such as automated AI-based scheduling, complete hospital management modules, or large-scale cloud deployment. It focuses mainly on providing a simple and effective operation scheduling solution that can be used in a real hospital environment with further improvements in the future.

Technologies and Tools Used

The development of the *Operation Scheduler for Hospital Management* system requires a combination of web technologies, database systems, and development tools to ensure smooth functionality and proper data management. Each technology used in this project plays a specific role in building a reliable and user-friendly application. The selected technologies are simple, efficient, and suitable for developing a web-based healthcare system within the internship environment.

The system is developed using front-end technologies to design the user interface, MongoDB to store and manage hospital data, and development tools to write, test, and maintain the project code. This combination ensures that the system is easy to develop, maintain, and scale in the future.

The main technologies and tools used in this project are explained below:

1. **HTML (HyperText Markup Language)**
   HTML is used to create the basic structure of the web application. It helps in designing web pages, forms, and input fields required for managing doctor details, patient information, and operation scheduling. HTML provides the layout of the system and ensures that all components are properly organized on the web pages.

Key uses:

- Creating login and registration pages
- Designing doctor and patient forms
- Structuring operation scheduling pages
- Organizing web content and layout

2. CSS (Cascading Style Sheets)

CSS is used to make the web application visually appealing and easy to use. It helps in designing the layout, colors, fonts, and overall appearance of the system. A well-designed interface improves user experience and makes the system easier to understand for administrators and users.

Key uses:

- Styling web pages and forms
- Designing dashboards and layouts
- Improving visual appearance
- Creating responsive design

3. JavaScript

JavaScript is used to add interactivity and functionality to the system. It handles user actions, validates form inputs, and manages dynamic data on the web pages. JavaScript also helps in connecting the front-end with the MongoDB database and ensures that data is processed correctly.

Key uses:

- Form validation and input handling
- Dynamic data updates
- Managing scheduling operations
- Handling user interactions

4. MongoDB

MongoDB is used as the main database for storing hospital data. It is a NoSQL database that stores information in the form of collections and documents, which makes it flexible and easy to manage. MongoDB allows efficient storage of doctor records, patient details,

operation schedules, and user data, ensuring fast data retrieval and secure data management.

Key uses:

- Storing doctor and patient information
- Managing operation scheduling data
- Maintaining user authentication data
- Ensuring structured and secure data storage
- Supporting fast data access and updates

5. Visual Studio Code

Visual Studio Code is used as the development environment for writing and managing the project code. It provides useful features like syntax highlighting, extensions, and debugging tools, which make coding easier and more organized.

Key uses:

- Writing and editing code
- Managing project files
- Debugging and testing
- Supporting extensions for development

6. GitHub

GitHub is used for version control and project management. It helps in storing the project code online and tracking changes during development. It also allows easy sharing of the project repository and proper documentation of the system.

Key uses:

- Storing project code
- Managing version control
- Maintaining README and documentation
- Sharing project repository

7. Web Browser (Chrome or Edge)

A web browser is used to run and test the application during development. It helps in checking the functionality, interface, and performance of the system to ensure that everything works properly.

Key uses:

- Running the web application
- Testing system features
- Debugging interface issues
- Monitoring performance

System Requirements

The *Operation Scheduler for Hospital Management* system is developed as a web-based application, so it requires a basic set of hardware and software to run smoothly. Since the system is designed for managing hospital operation scheduling, it should be able to handle data efficiently, run without performance issues, and provide a smooth experience for administrators and users. The system requirements define the minimum and recommended setup needed to develop, test, and run the application properly.

The requirements are divided into hardware and software components to ensure that the system works in a stable and reliable environment. These requirements are kept simple so that the application can run on a standard computer system without needing high-end infrastructure.

Hardware Requirements

The hardware requirements for this project are minimal because the system is lightweight and web-based. It can run on a normal laptop or desktop used for development and testing. A stable internet connection is important because the system uses MongoDB for database storage and GitHub for version control.

Minimum Hardware Requirements:

- Processor: Intel i3 or equivalent
- RAM: 4 GB
- Storage: 256 GB or higher
- Internet Connection: Required for database and project access
- Display: Standard laptop or desktop screen

Recommended Hardware Requirements:

- Processor: Intel i5 or higher
- RAM: 8 GB or more
- Storage: 512 GB SSD

- Internet Connection: Stable broadband connection
- Display: Full HD screen for better development and testing

These hardware specifications are enough to develop, run, and test the system efficiently without performance issues.

Software Requirements

The software requirements include the operating system, development tools, programming technologies, and database used to build the system. These tools help in writing code, managing data, testing the application, and maintaining the project.

Operating System:

- Windows 10 or above (preferred)
- Linux or macOS (optional)

Development Tools:

- Visual Studio Code for coding and project management
- GitHub for version control and repository management
- Web Browser (Google Chrome or Microsoft Edge) for running and testing the application

Programming Technologies:

- HTML for creating the structure of web pages
- CSS for designing and styling the interface
- JavaScript for adding functionality and interactivity

Database:

- MongoDB for storing hospital data such as doctor details, patient records, and operation schedules
- MongoDB Compass (optional) for managing and visualizing database collections

Additional Requirements:

- Internet connection for database connectivity and GitHub access
- Node.js (if backend or API integration is used)
- Basic server environment for running the web application

These software requirements ensure that the system can be developed and tested in a stable environment while maintaining proper performance and data management.

Functional Requirements

Functional requirements describe what the system is expected to do. These requirements define the main features and operations that the application must support to manage hospital operation scheduling effectively.

Key functional requirements include:

- User registration and login system
- Admin dashboard for managing hospital data
- Doctor management module
- Patient management module
- Operation scheduling and OT assignment
- Surgical report and operation data management
- Secure storage of data in MongoDB
- Easy access to operation and doctor information

Non-Functional Requirements

Non-functional requirements focus on the quality and performance of the system. These requirements ensure that the system is reliable, secure, and easy to use.

Key non-functional requirements include:

- User-friendly interface
- Fast data storage and retrieval
- Secure database management
- Reliable system performance
- Easy maintenance and scalability
- Proper data organization
- Smooth and responsive web interface

System Architecture

The system architecture of the *Operation Scheduler for Hospital Management* explains how different parts of the application work together to perform operation scheduling and data management. It provides a clear structure of how the user interface, application logic, and database interact with each other to make the system function properly. A well-planned architecture helps in maintaining system stability, improving performance, and making future updates easier.

The system follows a web-based architecture where users interact with the application through a browser. The front-end handles user input and displays information, while the application logic processes requests and communicates with the MongoDB database to store and retrieve data. This structured approach ensures that the system remains organized and efficient.

The architecture of the system is divided into three main layers:

1. Presentation Layer (Front-End)

This layer is responsible for the user interface of the system. It includes all the web pages and forms that users and administrators interact with. The front-end is built using HTML, CSS, and JavaScript to create a simple and easy-to-use interface.

Main responsibilities:

- Display login and registration pages
- Show admin dashboard and user panels
- Collect doctor and patient details
- Display operation schedules and reports
- Handle user interactions and inputs

This layer ensures that users can easily navigate through the system and perform required tasks without confusion.

2. Application Layer (Processing and Logic)

The application layer handles the core functionality of the system. It processes user requests, validates data, manages scheduling operations, and connects the front-end with the database. This layer ensures that all system operations are performed correctly and efficiently.

Main responsibilities:

- Process login and authentication
- Manage doctor and patient data
- Handle operation scheduling logic

- Assign operation theaters and surgery timings
- Validate user inputs
- Communicate with MongoDB database

This layer acts as the bridge between the user interface and the database, ensuring smooth data flow.

3. Database Layer (MongoDB)

The database layer stores all the hospital-related data in a structured format. MongoDB is used to manage doctor records, patient information, operation schedules, and user data. It allows fast data storage and retrieval, making the system reliable and scalable.

Main responsibilities:

- Store doctor and patient information
- Store operation schedules and surgical data
- Maintain user authentication records
- Provide fast data access and updates
- Ensure secure and structured data storage

This layer ensures that all hospital data is stored safely and can be accessed whenever required.

Working of the System Architecture

The overall working of the system can be explained in simple steps:

1. The user or administrator opens the web application in a browser.
2. The system displays the login or dashboard interface.
3. The user enters details such as doctor information, patient data, or operation schedule.
4. The application processes the input and validates the data.
5. The data is stored in the MongoDB database.
6. When required, the system retrieves data from MongoDB.
7. The updated information is displayed on the web interface.

This process ensures smooth communication between all parts of the system and allows efficient operation scheduling.

Advantages of the System Architecture

- Simple and easy to understand structure

- Efficient data handling using MongoDB
- Clear separation between front-end and database
- Easy to maintain and update
- Scalable for future development
- Reliable and secure data management

System Modules (Admin & User)

The *Operation Scheduler for Hospital Management* system is designed using a modular structure so that different users of the system can perform their tasks in an organized and controlled manner. Dividing the system into modules helps in managing responsibilities clearly, improving system security, and ensuring that each user has access only to the features they need. This approach also makes the system easier to maintain, update, and expand in the future.

In a hospital environment, different people perform different roles. Administrators are responsible for managing doctors, patients, and operation schedules, while other users mainly need access to view information related to operations and doctors. To handle these responsibilities efficiently, the system is divided into two main modules: the **Admin Module** and the **User Module**. Both modules are connected to the same database and work together to ensure smooth operation scheduling and proper data management.

Admin Module

The Admin Module is the core part of the system because it controls most of the hospital operation scheduling activities. The administrator is responsible for managing all important data such as doctor details, patient information, operation theater assignments, and surgical schedules. This module provides full access and control over the system, allowing the admin to add, update, monitor, and manage hospital operation records.

The admin dashboard is designed to be simple and user-friendly so that hospital staff can easily manage operation scheduling without needing advanced technical knowledge. All important operations can be handled from a single interface, which reduces manual workload and improves efficiency. The admin can continuously monitor hospital operations and update data whenever required, ensuring that the system remains accurate and up to date.

Main responsibilities of the Admin Module include:

- Secure admin login and authentication
- Access to the main dashboard

- Adding and managing doctor details
- Managing patient records and information
- Scheduling surgical operations
- Assigning operation theater (OT) numbers and timings
- Managing anesthesia and anesthesiologist details
- Recording required drugs and surgical instruments
- Updating surgical reports and operation remarks
- Monitoring operation schedules and hospital activities
- Editing or deleting outdated records
- Storing and retrieving data using MongoDB

Through this module, the administrator can maintain full control over the hospital operation scheduling system and ensure that all surgical activities are properly managed.

User Module

The User Module is designed for general users who need access to hospital operation-related information but do not require full control over the system. These users may include hospital staff, assistants, or authorized personnel who want to view doctor details and operation schedules. This module provides limited access to maintain system security and prevent unauthorized modifications to important data.

The user interface is kept simple and easy to navigate so that users can quickly access the information they need. Users can log in to the system, view doctor details, and check operation schedules without making any changes to the database. This helps in maintaining data integrity while still providing transparency and accessibility.

Main responsibilities of the User Module include:

- User registration and login
- Secure access to the system
- Viewing doctor information
- Viewing operation schedules
- Accessing surgical and OT details
- Checking available doctors and operation timings
- Navigating through system pages easily

This module ensures that users can stay informed about hospital operations while keeping the system secure and well-managed.

Interaction Between Admin and User Modules

The Admin and User modules are connected through the system database and application logic. The administrator updates and manages hospital data, and the same updated data becomes available to users in real time. MongoDB stores all information securely, ensuring that any changes made by the admin are reflected immediately in the user interface.

This modular approach makes the system more organized, secure, and efficient. It ensures proper coordination between hospital staff and improves overall operation scheduling management.

System Workflow

The system workflow explains how the *Operation Scheduler for Hospital Management* application works step by step. It describes the complete process starting from user login to operation scheduling and data management. A clear workflow helps in understanding how different components of the system interact and how hospital operation scheduling is carried out in an organized and systematic manner.

In this system, the workflow is designed to be simple and efficient so that hospital staff can perform their tasks without confusion. The system ensures smooth communication between the user interface, application logic, and MongoDB database, allowing data to move seamlessly across different modules.

Step-by-Step Working of the System

The workflow of the system follows a structured sequence of operations:

1. The administrator or user opens the web application using a browser.
2. The system displays the login or registration page.
3. The user enters login credentials to access the system.
4. The system verifies the credentials and allows access to the dashboard.
5. The administrator enters the admin panel to manage hospital data.
6. Doctor details are added and stored in the system.
7. Patient information is recorded and saved in MongoDB.
8. The administrator schedules a surgical operation.
9. Operation theater (OT) number and surgery timing are assigned.
10. Anesthesia and anesthesiologist details are added.
11. Required drugs and surgical instruments are recorded.

12. Surgical reports and operation remarks are updated.
13. All operation data is stored securely in MongoDB.
14. Users log in to view doctor and operation information.
15. The system retrieves data from the database and displays it on the interface.
16. The administrator monitors and updates operation schedules when required.

This step-by-step workflow ensures that hospital operations are managed in a smooth and organized manner.

Workflow Stages

To make the process easier to understand, the system workflow can be divided into three main stages:

1. Login and Authentication Stage

- User or admin logs into the system
- Credentials are verified by the system
- Access to the dashboard is granted

2. Operation Management Stage

- Admin manages doctor and patient data
- Surgical operations are scheduled
- OT numbers and surgery timings are assigned
- Surgical and operation details are recorded

3. Monitoring and Data Access Stage

- Data is stored in MongoDB
- Users view doctor and operation details
- Admin monitors and updates schedules
- System maintains real-time data availability

Benefits of the System Workflow

- Clear and structured operation process
- Reduced manual work in hospital scheduling
- Faster and more accurate data handling
- Real-time access to operation schedules
- Secure data storage using MongoDB

- Better coordination between hospital staff

Database Design (MongoDB)

The database is one of the most important components of the *Operation Scheduler for Hospital Management* system because it is responsible for storing and managing all the hospital-related information in an organized and secure manner. Since the system deals with sensitive and structured data such as doctor details, patient records, operation schedules, and surgical information, it requires a reliable and flexible database that can handle data efficiently. For this project, **MongoDB** is used as the primary database because of its simplicity, scalability, and ability to manage large amounts of data in a structured way.

MongoDB is a NoSQL database that stores data in the form of collections and documents instead of traditional tables and rows. This makes it more flexible and easier to work with in web-based applications. In a hospital operation scheduling system, data may change frequently, such as doctor availability, patient records, and operation schedules. MongoDB allows these changes to be handled smoothly without affecting the overall system structure. It also provides faster data retrieval and easier integration with web technologies, which makes it suitable for this project.

The database is designed in a way that all hospital operation-related data is connected and organized properly. Each module of the system interacts with MongoDB to store and retrieve information whenever required. When the administrator adds doctor details or schedules an operation, the data is stored in MongoDB. When users access the system, the database retrieves the required information and displays it on the interface. This continuous interaction ensures real-time data availability and smooth system performance.

Purpose of Using MongoDB

MongoDB was selected for this project because it provides flexibility and efficiency in handling healthcare data. Unlike traditional databases, it allows developers to store structured and semi-structured data without complex table relationships. This makes the system easier to develop, maintain, and expand in the future.

The main reasons for using MongoDB in this project include:

- Flexible data structure for hospital records
- Easy storage of doctor, patient, and operation information
- Faster data retrieval and updates
- Scalable database for future expansion
- Secure and organized data management
- Easy integration with web-based systems

- Efficient handling of real-time scheduling data

These advantages make MongoDB a suitable and practical choice for developing the operation scheduling system.

Database Collections

In MongoDB, data is stored in collections, and each collection represents a specific part of the system. The database of this project is divided into different collections to maintain proper organization and easy data access. Each collection stores related information and connects with other collections to support operation scheduling.

The main collections used in the system are:

Admin Collection
This collection stores administrator login details and access control information. It ensures that only authorized administrators can manage hospital operation scheduling and system data.

User Collection
This collection stores user registration and login information. It allows authorized users to access the system and view operation-related details securely.

Doctor Collection
This collection contains all doctor-related information such as doctor name, specialization, contact details, and availability. It helps in assigning doctors to specific operations and managing their schedules.

Patient Collection
This collection stores patient details including name, age, gender, contact information, and medical data. It ensures that patient records are maintained properly for surgical scheduling.

Operation Collection
This is the most important collection in the system because it stores operation scheduling data. It connects doctors and patients and includes information such as operation theater number, surgery timing, anesthesia details, required drugs, and surgical remarks.

All these collections work together to ensure that hospital operation data is stored and managed efficiently.

Data Flow in MongoDB

The system continuously interacts with MongoDB whenever data is entered, updated, or viewed. This interaction helps in maintaining real-time data and ensures smooth system operation.

The basic data flow works as follows:

1. The administrator enters doctor or patient details into the system.
2. The data is sent to MongoDB and stored in the respective collection.

3. Operation scheduling details are added and connected with doctor and patient records.
4. MongoDB stores the complete operation information securely.
5. When users request information, the database retrieves the required data.
6. The system displays the data on the web interface.

This process ensures that all hospital data remains updated and easily accessible.

Advantages of MongoDB Database Design

The MongoDB database design provides several benefits to the system and improves overall performance.

- Flexible and easy-to-manage data structure
- Fast storage and retrieval of hospital records
- Scalable for future system expansion
- Secure data handling and management
- Real-time data access and updates
- Efficient handling of operation scheduling data

ER Diagram

The ER (Entity Relationship) Diagram provides a clear and structured representation of how different entities in the *Operation Scheduler for Hospital Management* system are connected to each other. It helps in understanding the relationship between doctors, patients, administrators, users, and operation scheduling data. The ER diagram acts as a blueprint of the database structure and shows how information flows within the system.

Even though MongoDB is a NoSQL database, the ER diagram is still useful in designing the relationships between different collections and entities. It helps in organizing data properly and ensures that all components of the system are logically connected. By using an ER diagram, developers and system designers can easily understand how hospital data is stored and managed.

Main Entities in the ER Diagram

The system includes several important entities that work together to manage operation scheduling and hospital data.

Admin Entity
The admin entity represents the administrator who controls the system. The admin is

responsible for managing doctors, patients, and operation schedules. This entity includes details such as admin ID, username, password, and access role.

User Entity
The user entity represents authorized users who can log in to the system and view operation-related information. This entity includes user ID, name, email, and password.

Doctor Entity
The doctor entity stores information about doctors working in the hospital. It includes doctor ID, doctor name, specialization, and contact details. Doctors are assigned to operations and connected to patient records.

Patient Entity
The patient entity stores patient-related information such as patient ID, name, age, gender, and medical details. Patients are linked to operations through the scheduling system.

Operation Entity
The operation entity is the central part of the ER diagram. It connects doctors and patients and stores operation scheduling data such as operation ID, OT number, surgery date and time, anesthesia details, required drugs, and surgical remarks.

Relationships Between Entities

The ER diagram shows how these entities interact with each other to form a complete hospital operation scheduling system.

The main relationships include:

- The admin manages doctors, patients, and operations
- Users can view doctor and operation information
- Doctors perform operations on patients
- Patients are assigned to operations
- The operation entity connects doctor and patient records
- MongoDB stores and manages all entity data

These relationships ensure proper coordination between all components of the system and help maintain structured data management.

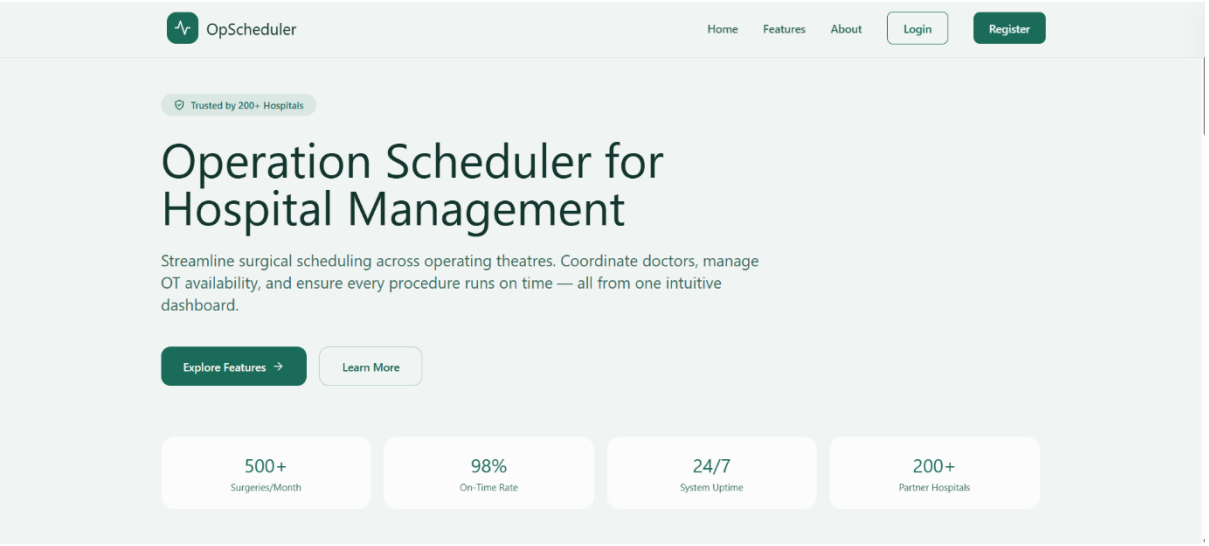
Importance of ER Diagram

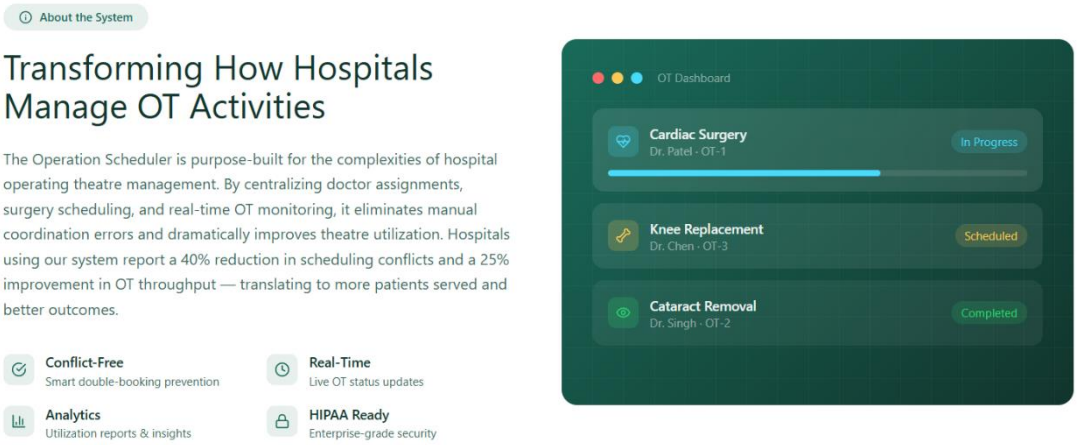
The ER diagram plays an important role in system development and database design because it provides a clear visual understanding of the system structure.

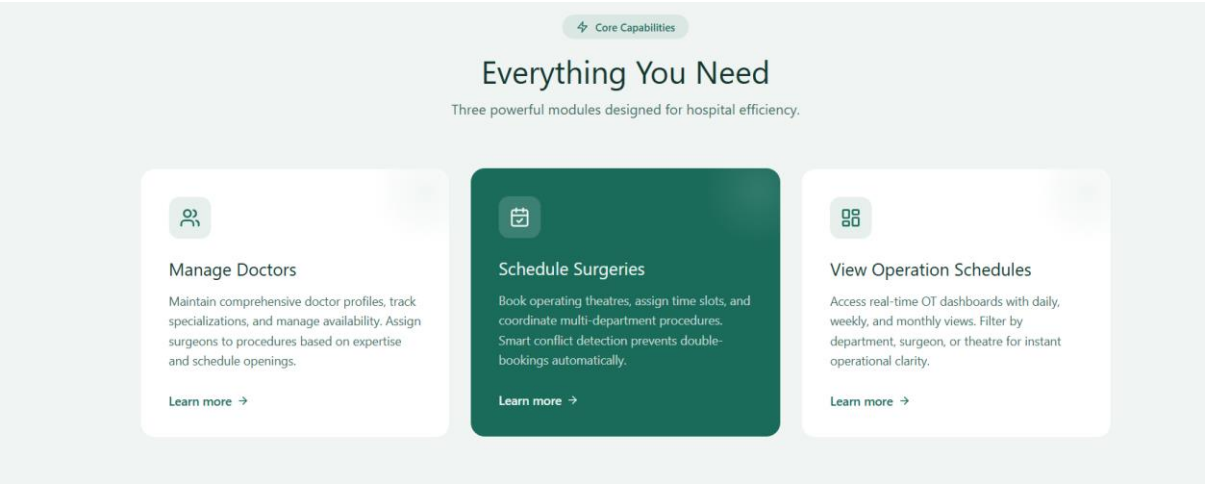
- Helps in understanding database structure
- Shows relationships between different entities
- Improves data organization and system clarity

- Supports proper system development and implementation
- Makes database design easier to manage
- Helps in maintaining structured data flow

Interface Design / Screenshots







Implementation Details

The implementation phase is the stage where the *Operation Scheduler for Hospital Management* system is developed and turned into a working web application. In this phase, the system design and database structure are converted into actual code, and all modules are integrated to create a functional and efficient system. The implementation focuses on building a stable, secure, and easy-to-use application that can manage hospital operation scheduling effectively.

The system is implemented using HTML, CSS, and JavaScript for the front-end, while MongoDB is used for storing and managing all hospital data. The development process follows a structured approach so that each module is created step by step and then integrated into the complete system. This ensures that the application runs smoothly and performs all required functions properly.

Development Approach

The development of the system follows a step-by-step approach to ensure proper functionality and stability. Each part of the system is developed separately and then connected to form a complete application.

The main steps followed during implementation include:

1. Designing the layout and structure of web pages
2. Creating login and registration functionality
3. Developing the admin dashboard and user module
4. Implementing doctor and patient management features
5. Building the operation scheduling system
6. Connecting the system with MongoDB database
7. Testing the application and fixing errors
8. Finalizing the system for deployment

This step-by-step development process helps in maintaining system quality and reducing errors during implementation.

Front-End Implementation

The front-end of the system is responsible for creating the user interface and handling user interactions. HTML is used to structure web pages, CSS is used for styling and layout, and JavaScript is used to add functionality and interactivity. Together, these technologies create a responsive and interactive web application.

The front-end implementation includes:

- Designing login and registration pages
- Creating admin dashboard and navigation system
- Building doctor and patient management forms
- Developing operation scheduling interface
- Handling user input and validation
- Displaying operation and doctor data

This ensures that users can easily interact with the system and perform required operations smoothly.

Database Implementation (MongoDB)

MongoDB is integrated into the system to store all hospital-related data in a structured and secure manner. The database manages doctor details, patient records, operation schedules, and user login information. Proper database design ensures that data is stored efficiently and can be retrieved quickly when required.

The database implementation includes:

- Creating collections for admin, user, doctor, patient, and operation
- Storing hospital data in structured documents
- Managing operation scheduling records
- Handling data retrieval and updates
- Ensuring secure data storage

This helps the system maintain real-time data and efficient performance.

System Integration

After developing the front-end and database components, all modules are integrated to form a complete working system. The interface communicates with MongoDB through application logic, ensuring smooth data flow between different components.

System integration includes:

- Connecting front-end with MongoDB database
- Linking admin and user modules
- Managing data communication between components
- Testing system functionality
- Ensuring smooth operation scheduling

This integration ensures that the system works as a unified and reliable application.

Key Features Implemented

The implementation of the system includes several important features that support hospital operation scheduling:

- Secure login and authentication system
- Admin and user modules
- Doctor and patient management system
- Operation scheduling module
- MongoDB database integration
- User-friendly interface
- Real-time data management

Importance of Implementation

The implementation phase ensures that the system becomes a functional application that can be used in real-world environments. It converts theoretical design into a practical solution and ensures that hospital operation scheduling can be managed efficiently.

The main benefits of proper implementation include:

- Smooth system functionality
- Reliable data management
- Efficient operation scheduling
- Improved system performance
- Support for future development and upgrades

GitHub Repository and Code Structure

Testing and Test Cases

Testing played a very important role in the development of the *Operation Scheduler for Hospital Management* system, as it ensured that every feature of the application worked correctly and met the required objectives. After completing the implementation phase, the system was carefully tested to verify that all modules such as login, doctor management, patient management, operation scheduling, and MongoDB database integration were functioning smoothly. The main goal of testing was to make sure that the system was reliable, error-free, and ready for real-world use in a hospital environment.

The testing process was carried out in a step-by-step manner, where each module was checked individually before testing the entire system together. This approach helped in identifying errors at an early stage and made it easier to fix them without affecting other parts of the system. Special attention was given to database connectivity, operation scheduling logic, and user authentication, as these were the core components of the application. By performing proper testing, the system was able to handle data efficiently and provide accurate results without any major issues.

To ensure the quality of the system, different types of testing were performed. Functional testing was done to verify that all features were working as expected, system testing was carried out to check the complete application, and database testing ensured that MongoDB was storing and retrieving data properly. The user interface was also tested to make sure that forms, buttons, and pages were easy to use and responsive. Overall, testing helped in improving the performance and stability of the application.

Types of Testing Performed

The following types of testing were conducted during the development of the system:

1. **Functional Testing** – Checked whether all modules such as login, doctor management, patient management, and operation scheduling were working properly.
2. **System Testing** – Verified that the complete system was functioning smoothly after integrating all modules.
3. **Database Testing** – Ensured that MongoDB stored and retrieved doctor, patient, and operation data correctly.
4. **User Interface Testing** – Verified that the system interface was user-friendly and easy to navigate.
5. **Authentication Testing** – Ensured that only authorized users could log in and access the system.

Test Cases

Several test cases were designed to validate the functionality of the system and ensure that all operations were performed correctly.

Test Case 1: Login with Valid Credentials

- Input: Correct username and password
- Expected Result: User logs into the dashboard successfully
- Status: Passed

Test Case 2: Login with Invalid Credentials

- Input: Incorrect username or password
- Expected Result: Error message is displayed
- Status: Passed

Test Case 3: Add Doctor Information

- Input: Doctor name, specialization, contact details
- Expected Result: Doctor data stored in MongoDB database
- Status: Passed

Test Case 4: Add Patient Information

- Input: Patient name, age, and medical details
- Expected Result: Patient data saved successfully
- Status: Passed

Test Case 5: Schedule Operation

- Input: Doctor, patient, OT number, date, and time
- Expected Result: Operation scheduled and stored in database
- Status: Passed

Test Case 6: View Operation Details

- Input: Request operation records
- Expected Result: Operation details displayed correctly
- Status: Passed

Test Case 7: Database Storage Verification

- Input: Operation data entered into system
- Expected Result: Data stored and retrieved from MongoDB correctly

- Status: Passed

Through these test cases, it was confirmed that the system performs all required operations accurately and efficiently. Testing ensured that the application was stable and capable of handling hospital operation scheduling tasks without errors.

Challenges Faced

While developing the *Operation Scheduler for Hospital Management* system, several challenges were encountered during different stages of the project. These challenges were part of the learning process and helped in gaining practical experience in building a real-world healthcare application. Overcoming these difficulties required patience, research, testing, and continuous improvement of the system.

One of the major challenges was designing a proper database structure using MongoDB. Since the system needed to store and manage doctor details, patient records, and operation schedules, it was important to organize the data in a structured and efficient manner. Initially, managing collections and ensuring proper relationships between different data entities was difficult, but with proper planning and ER diagram design, the database structure was improved.

Another challenge was integrating MongoDB with the web application. Establishing a stable connection between the front-end and the database required understanding how MongoDB works and how data can be stored and retrieved efficiently. Multiple tests and configuration changes were required to ensure smooth database connectivity and proper data handling.

The operation scheduling logic was also challenging to implement because it involved coordinating multiple entities such as doctors, patients, and operation theaters. Ensuring that there were no scheduling conflicts and that operation details were stored correctly required careful planning and testing. Developing a simple and user-friendly interface while maintaining system functionality was another challenge, as the interface needed to be clear and easy to use for hospital staff.

Major Challenges Encountered

The main challenges faced during the development of the project include:

1. Designing a structured MongoDB database for hospital data
2. Establishing proper MongoDB integration with the web application
3. Implementing operation scheduling logic without conflicts
4. Developing a secure login and authentication system
5. Creating a clean and user-friendly interface
6. Handling data validation and error management

7. Testing and debugging different system modules

Solutions Implemented

To overcome these challenges, several solutions were applied during development:

- Proper planning was done before designing the MongoDB database
- Multiple testing sessions were conducted to ensure database connectivity
- Operation scheduling logic was simplified and optimized
- Login validation and authentication checks were implemented
- Interface design was improved through continuous adjustments
- Regular debugging helped in fixing system errors

Learning Outcomes from Challenges

Facing these challenges provided valuable learning and practical experience during the internship.

- Improved understanding of MongoDB and database management
- Better knowledge of web application development
- Practical experience in system testing and debugging
- Enhanced problem-solving and analytical skills
- Better understanding of real-world healthcare systems

Future Enhancements

The *Operation Scheduler for Hospital Management* system has been developed to provide a structured and efficient way of managing operation theater scheduling and hospital surgical activities. While the current version of the system successfully fulfills its core objectives, there are several areas where the system can be further improved in the future. These improvements can make the application more practical for real-world hospital environments and enhance its overall performance, usability, and scalability.

At present, the system mainly focuses on managing doctors, patients, and operation scheduling through a web-based interface supported by MongoDB for data storage. In the future, the system can be expanded to include additional hospital management features so that it becomes a more complete healthcare solution. For example, integrating other hospital departments such as pharmacy, laboratory, billing, and emergency services would allow all hospital operations to be managed within a single platform. This would reduce the need for multiple systems and improve coordination between different departments.

Another important enhancement could be the introduction of real-time notifications and alerts. Doctors and administrators could receive notifications about scheduled surgeries, changes in operation timing, or emergency cases. This would help in improving communication and ensuring that all staff members remain updated about important activities. Similarly, adding an online appointment and surgery booking system would allow patients or hospital staff to schedule operations more conveniently, reducing manual effort and improving workflow efficiency.

The system can also be improved by introducing cloud-based deployment and mobile accessibility. With cloud integration, hospital staff would be able to access the system from different locations, and a mobile application could allow doctors and administrators to manage operations directly from their smartphones. This would increase flexibility and make the system more accessible in real-time hospital environments.

In addition, advanced data analytics and reporting features can be added to help hospital management analyze surgical performance, resource utilization, and operation trends. Such insights would support better decision-making and help hospitals improve their operational efficiency. Security enhancements, including stronger authentication and encrypted data storage, can also be implemented to ensure that sensitive patient and surgical data remain protected.

Suggested Future Improvements

The following enhancements can be considered for future development:

1. Integration with a complete hospital management system
2. Real-time notifications and operation alerts
3. Online appointment and surgery booking system
4. Emergency operation scheduling module
5. Cloud-based system deployment
6. Mobile application for doctors and administrators
7. Advanced data analytics and reporting dashboard
8. Role-based access control for different hospital staff
9. Improved security and data encryption
10. AI-based operation scheduling and resource optimization

These enhancements would make the system more powerful, flexible, and suitable for real-world healthcare environments, while also improving efficiency and user experience.

Conclusion

The *Operation Scheduler for Hospital Management* project was developed with the goal of simplifying and automating the operation theater scheduling process in hospitals. Managing surgical operations manually can be time-consuming and prone to errors, which often leads to scheduling conflicts, poor coordination, and inefficient use of hospital resources. This project successfully addresses these issues by providing a structured and web-based solution that allows administrators to manage doctors, patients, and operation schedules in an organized and efficient manner.

Throughout the internship, the focus was on designing and developing a system that is simple, reliable, and easy to use. The project was built using HTML, CSS, and JavaScript to create a clean and user-friendly interface, while MongoDB was used as the database to store and manage hospital data efficiently. The use of a modular system design made it easier to develop and test different components of the application, ensuring that each part of the system works properly and integrates smoothly with the others.

The project achieved its main objectives by automating operation scheduling, reducing manual work, improving coordination between hospital staff, and ensuring secure data storage. Proper system testing and debugging helped in making the application stable and reliable, while the structured deployment approach ensured that the system is ready for real-world use. The implementation of MongoDB allowed efficient data handling and improved overall system performance, making the application scalable and adaptable for future improvements.

Working on this project during the internship provided valuable practical experience in real-world software development. It helped in developing a better understanding of system architecture, database design, web development, and testing processes. The project also improved problem-solving skills, logical thinking, and the ability to design structured and efficient software solutions. Overall, this experience contributed significantly to both technical knowledge and professional development.

Key Outcomes of the Project

1. Developed a web-based operation scheduling system for hospitals
2. Implemented doctor and patient management modules
3. Integrated MongoDB for structured and efficient data storage
4. Designed a simple and user-friendly interface
5. Created a modular and scalable system structure
6. Successfully tested and prepared the system for deployment
7. Gained practical experience in healthcare software development

In conclusion, the *Operation Scheduler for Hospital Management* system demonstrates how modern web technologies and database solutions can be used to improve hospital operation

scheduling and management. The project not only fulfills the internship requirements but also provides a strong foundation for future development and real-world healthcare system implementation. It reflects the practical application of technical skills learned during the internship and highlights the importance of structured and automated solutions in modern healthcare management.

References

1. MongoDB Official Documentation (Google)
2. MDN Web Docs (HTML, CSS, JavaScript)
3. W3Schools Web Development Resources
4. International Journal of Advance and Innovative Research – Patient Scheduling System (2025)
5. IJCRT – Hospital Management System with MongoDB
6. Jurnal Sistem Informasi – MongoDB Implementation in Scheduling System

GitHub Repository Link